按测量结果定位瓶颈. 保持一个观看者，等至少 3 秒后读 `/status` 或看串口的那一行：`jpeg` 最大，瓶颈在 ARM 上的图像处理，先试 `-s`；`capture` 最大，检查 CSI/DMA 是否正常、帧是否靠 watchdog 而不是中断取得；`send` 最大，检查 Wi-Fi 信号、TCP 发送队列，或是否有一个慢客户端拖住了所有人。

还可以继续做的. 下面几项目前没有实现，按预期收益排列：

- **把 JPEG 拆到多个核上**：编码是按 16 行的 MCU 行进行的，可以把一帧分成上下两半交给两个 JPEG 线程各自编码，再把两段熵编码结果用 restart marker（`RST`*n*）拼接；Pi 3 有四个核，而现在只用上了三个执行流；
- **用查表和定点优化热点**：去马赛克和色彩转换对每个像素都做多次乘除，可以把 gamma、白平衡增益合并进一张查表，色彩空间转换改用移位和加法；
- **让发送不阻塞流水线**：给每个观看者独立的发送队列，慢客户端只丢自己的帧，不再拖慢别人；
- **可缓存的 DMA 映射**：现在为了正确性用 `Normal-NC`，CPU 读 384 KB 像素时每次都要访问 DRAM。改成可缓存映射可以加快 JPEG 一段，但必须在每个槽交还给用户前严格作废对应的缓存行（7.9.8 小节）。

7.9.14 当前状态

真机（Pi 3B+，Camera v1.3）上，v8 驱动从第一次访问起就用 25 kHz 软件 SCCB，稳定读到芯片 ID `0x5647` 并绑定到 `/dev/video0`；Unicam 以轮询方式拿到完整帧（单缓冲版本），`camshot -t 1 -s -o /boot/cam02.bmp` 能把彩条写成 BMP（7.9.10 小节）。驱动已经改为连续出流、DMA 双缓冲、以 CSI1 中断通知帧完成，并支持零拷贝 `mmap`（7.9.8 小节）；`camshot -v` 可以连续录成一串 JPEG，`camserver` 可以通过 HTTP 推送 MJPEG（7.9.12 小节）。还没有完成的工作有：

- 在真机上确认 CSI1 中断路径（日志 `frame captured by CSI1 IRQ 39`），watchdog 轮询保留为兜底；
- 查清 BSC0 硬件读不到芯片 ID 的原因（控制器路由还是固件占用），以便退回硬件 I2C；
- 更多模式：Linux 的 1296×972 等模式每帧超过 1.5 MB，需要扩大 `CAMDMA`，桥接驱动不用改；目前也还没有选择模式的接口；
- 只有两个槽，没有 `poll()`，也没有手动曝光；`camserver` 已改为三级流水线，JPEG 编码（约 308 ms/帧）是目前的瓶颈（7.9.13 小节）；
- `arasan_sdio.c`、`dwc2.c`、`sd.c` 里仍各有一份私有 mailbox 代码，应统一到 `mbox.c`；
- `config.txt` 不能打开固件的摄像头支持（`start_x=1`、`camera_auto_detect=1`），否则固件会占用同一个传感器和 I2C。

注 摄像头驱动是全书最能体现“分层”价值的例子：mailbox、I2C、时钟、电源域、CSI-2 接收器、DMA、帧状态机、字符设备、用户态图像处理，每一层都可以单独验证。真机调试也正是这样一层层推进的：先看 mailbox 回读，再用 bit-bang 判断 I2C 故障在控制器内外，再用传感器内部彩条（`camshot -t 1`）验证 CSI-2 和 DMA，最后才看真实画面。